

Title: AI-Based Crop Yield Prediction and Agricultural Production Analysis

Project Report

VT Rushi Kannan

4th Year



Fundamentals of AI Using Agriculture Data Set

Certificate

This is to certify that VT Rushi Kannan from Koneru Lakshmaiah University has successfully completed the project work titled **AI-Based Crop Yield Prediction and Agricultural Production Analysis** as part of the 1-credit course “Fundamentals of AI using Agriculture Data Set”, offered under ANNAM.AI – Centre of Excellence, Ministry of Education, Government of India, at IIT Ropar. This project constitutes a component of the 30 hours of learning prescribed for the course and reflects the student’s independent effort in problem selection, method design, and implementation.

Index

1. Introduction and Motivation
2. Problem Statement
3. Dataset Understanding
4. Methodology
5. Implementation Details
6. Results and Discussions
7. Conclusion
8. References

Appendix

1. Introduction and Motivation

Agriculture is one of the most important sectors of the Indian economy, providing livelihoods to millions of people and ensuring food security across the country [1]. Farmers and agricultural planners make important decisions based on factors such as cultivated area, crop production, yield, and seasonal conditions. Analysing historical agricultural data can help identify trends and support better planning for improved productivity. Recent studies have shown that Artificial Intelligence and Machine Learning techniques can effectively analyse agricultural data and improve crop yield prediction [3]–[7].

In this project, the All-India Crop-wise Area, Production & Yield dataset published by the Department of Agriculture & Farmers Welfare, Government of India was used to analyse historical agricultural data [1]. Artificial Intelligence and Machine Learning techniques were applied to identify patterns and develop a model for predicting crop yield. The insights obtained from this analysis can support farmers, researchers, and policymakers in making informed decisions and demonstrate the practical application of AI in agriculture [4], [5].

2. Problem Statement

Crop yield varies across different crops, seasons, and years, making it challenging to analyse agricultural trends and accurately estimate future outcomes. Previous studies have demonstrated that Machine Learning techniques can effectively model agricultural data and improve crop yield prediction by identifying complex patterns in historical datasets [3]–[7]. This project aims to analyse historical crop-wise area, production, and yield data and develop a Machine Learning model to predict crop yield. The expected outcome is to provide data-driven insights that can support better agricultural planning and decision-making for farmers, researchers, and policymakers [1], [4].

3. Dataset Understanding

The project uses the All-India Crop-wise Area, Production & Yield dataset published by the Department of Agriculture & Farmers Welfare, Government of India [1]. The dataset consists of 98 records and 17 columns, containing crop-wise agricultural statistics from 2021–22 to 2025–26. It includes information on 34 different crops across four growing seasons: Kharif, Rabi, Summer, and Total.

The main features in the dataset include Crop, Season, Area, Production, and Yield for each year. During the initial analysis, it was observed that only the 2021–22 Area, Production, and

Yield columns contained missing values, while no duplicate records were found. The missing values were handled during data preprocessing, and the cleaned dataset was used to analyse agricultural trends and develop a Machine Learning model for crop yield prediction. The dataset provides a reliable foundation for applying AI and Machine Learning techniques to agricultural data analysis and prediction [1], [3].

4. Methodology

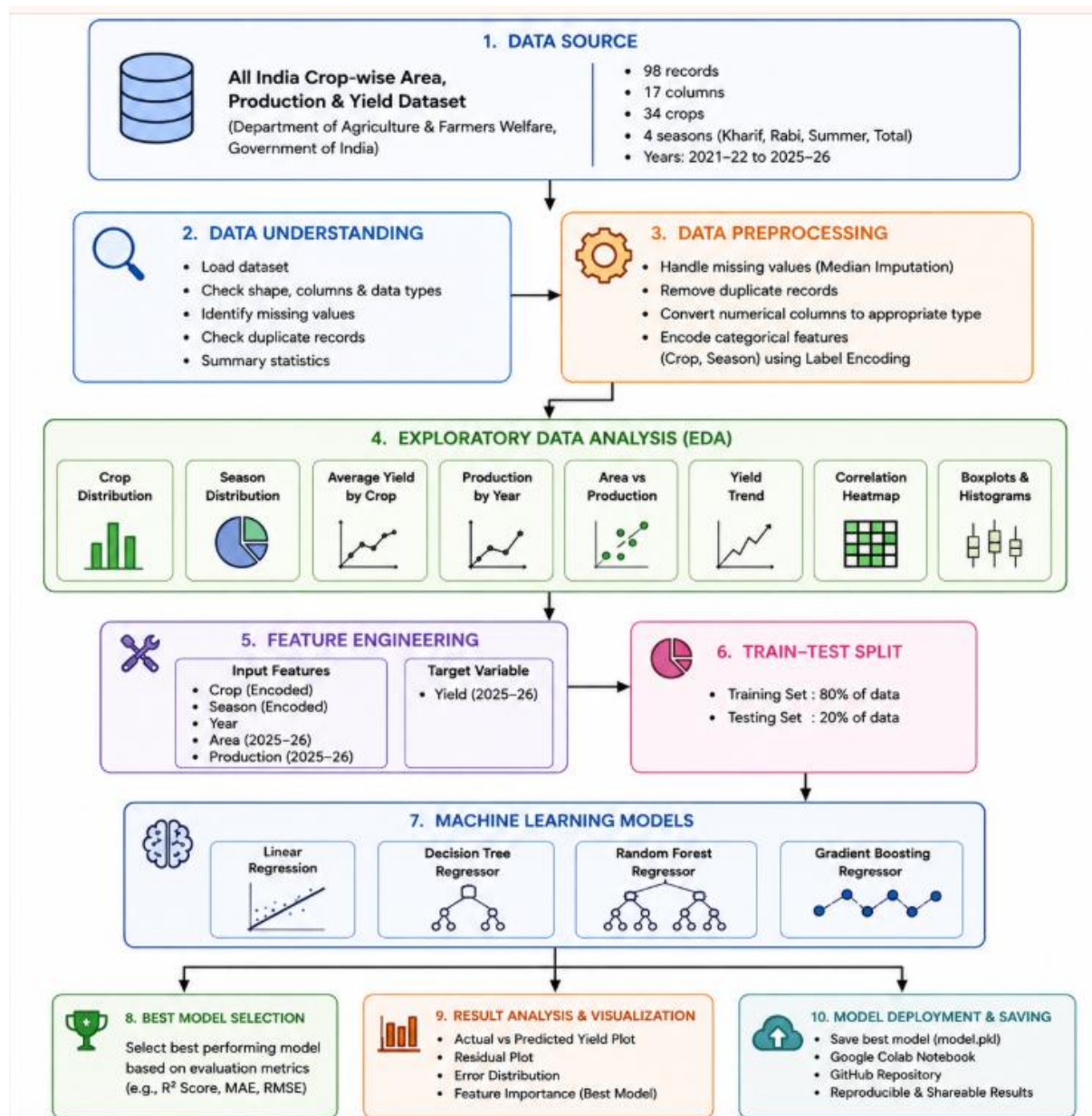


Figure-1 End-to-End Machine Learning Framework for Crop Yield Estimation

The project followed a structured Machine Learning workflow to analyse agricultural data and predict crop yield, as illustrated in Figure 1. First, the dataset was loaded and examined to understand its structure, data types, missing values, and overall quality (Figure 1, Steps 1–2). Data preprocessing was then performed by handling missing values via median imputation, removing duplicate records, converting numerical columns, and encoding categorical features (*Crop* and *Season*) using Label Encoding (Figure 1, Step 3).

After preprocessing, Exploratory Data Analysis (EDA) was carried out to examine the distribution of crops and seasons, analyse temporal trends in area, production, and yield, and identify relationships between key variables through visualization (Figure 1, Step 4). For model development, the relevant input features (*Crop*, *Season*, *Year*, *Area-2025-26*, and *Production-2025-26*) were selected alongside the target variable, *Yield-2025-26* (Figure 1, Step 5).

The pre-processed dataset was partitioned into an 80% training set and a 20% testing set (Figure 1, Step 6). Four Machine Learning algorithms—Linear Regression, Decision Tree Regressor, Random Forest Regressor, and Gradient Boosting Regressor—were trained and evaluated across standard evaluation metrics, including R^2 Score, MAE, MSE, and RMSE (Figure 1, Step 7). Finally, the best-performing model was selected and exported for downstream deployment (Figure 1, Steps 8–9).

5. Implementation Details

The project was implemented using Python in Google Colab. The dataset was loaded using the Pandas library, while NumPy was used for numerical operations. Data visualization was performed using Matplotlib to understand crop distribution, seasonal patterns, production trends, and yield variations. Data preprocessing included handling missing values, converting numerical columns, and encoding the categorical features *Crop* and *Season* using Label Encoding.

The Machine Learning models were developed using the Scikit-learn library. The dataset was divided into 80% training data and 20% testing data, and four regression models—Linear Regression, Decision Tree Regressor, Random Forest Regressor, and Gradient Boosting Regressor—were trained and evaluated. Their performance was compared using R^2 Score, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). The best-performing model was then saved using the Joblib library for future use. The generated visualizations and trained model were also stored in Google Drive for documentation and future reference.

Google Colab Link:

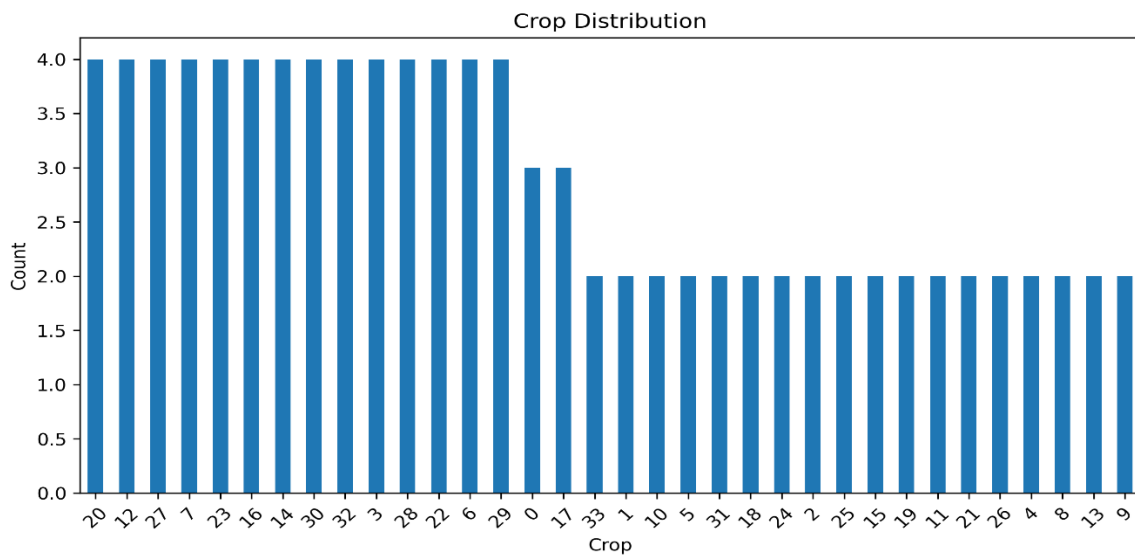
<https://colab.research.google.com/drive/1lJsWDK4Zial4Zf5eyzKTABF7TesPyQYG?usp=sharing>

Git-Hub Repository Link:

https://github.com/Rushikannan2/AI_Agriculture_Project

6. Results and Discussions

6.1 Exploratory Data Analysis Results



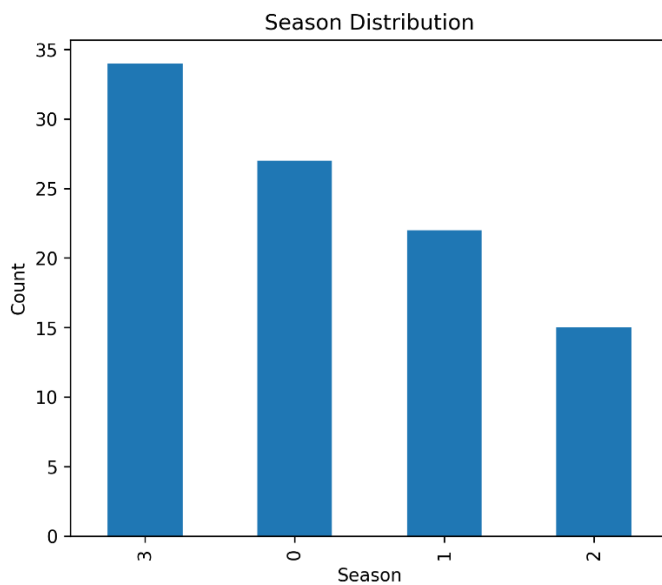


Figure-3 Season Distribution

Figure 3 shows the distribution of records across the four agricultural seasons, indicating that the Total season has the highest number of records, followed by Kharif, Rabi, and Summer (encoded as 0 = Kharif, 1 = Rabi, 2 = Summer, and 3 = Total).

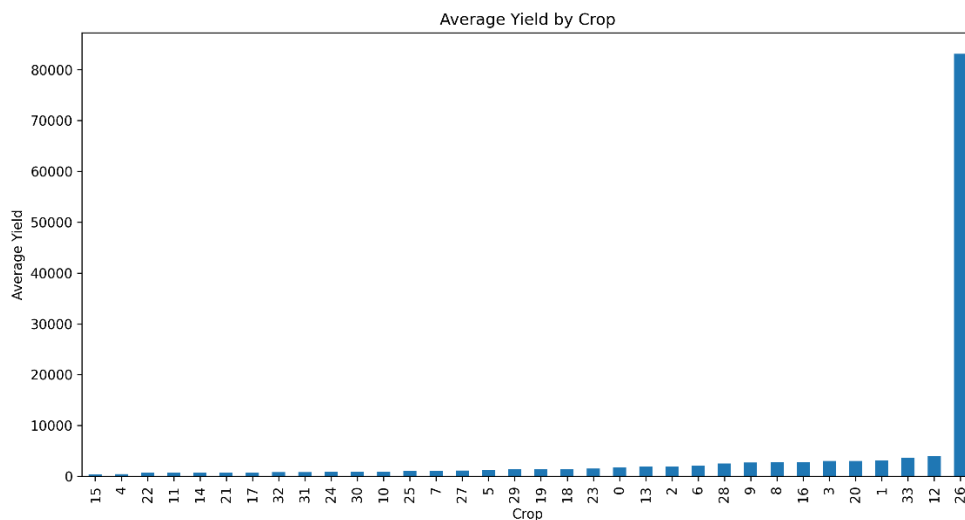


Figure-4 Average Yield by Crop

Figure 4 shows the average crop yield across different crops, indicating that crop type significantly influences agricultural productivity, with some crops consistently achieving higher yields than other

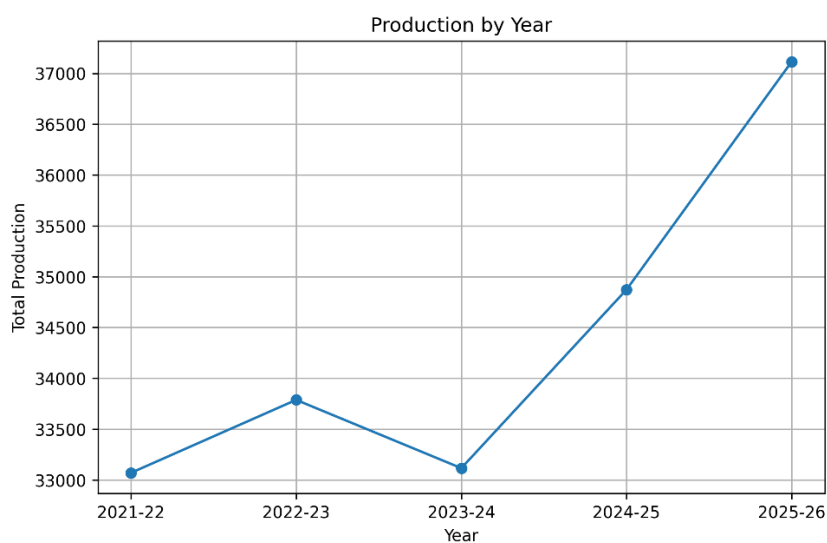


Figure-5 Production by Year

Figure 5 shows the yearly crop production trend, indicating an overall increase from 2021–22 to 2025–26 despite a slight decline in 2023–24, reflecting gradual growth in agricultural production over the study period.

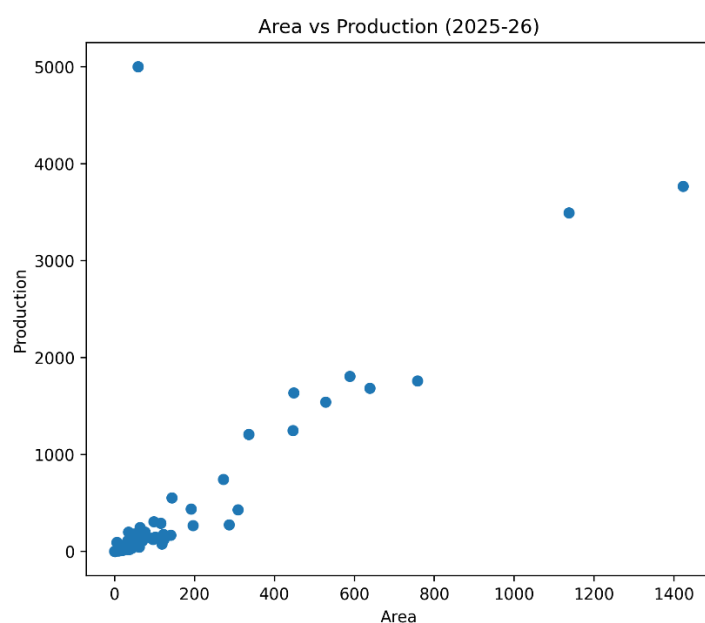


Figure-6 Area vs Production

Figure 6 shows the relationship between cultivated area and crop production, indicating that larger cultivation areas generally result in higher crop production.

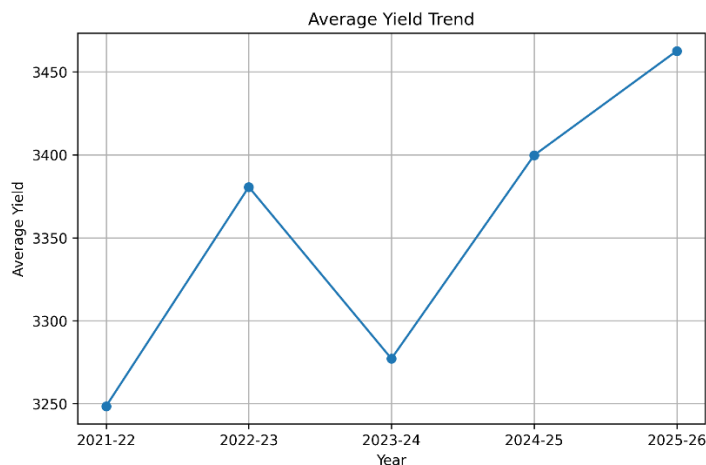


Figure-7 Yield Trend

Figure 7 shows the average crop yield changes across different years, with an overall increasing trend toward 2025–26. This indicates an improvement in agricultural productivity over the study period.

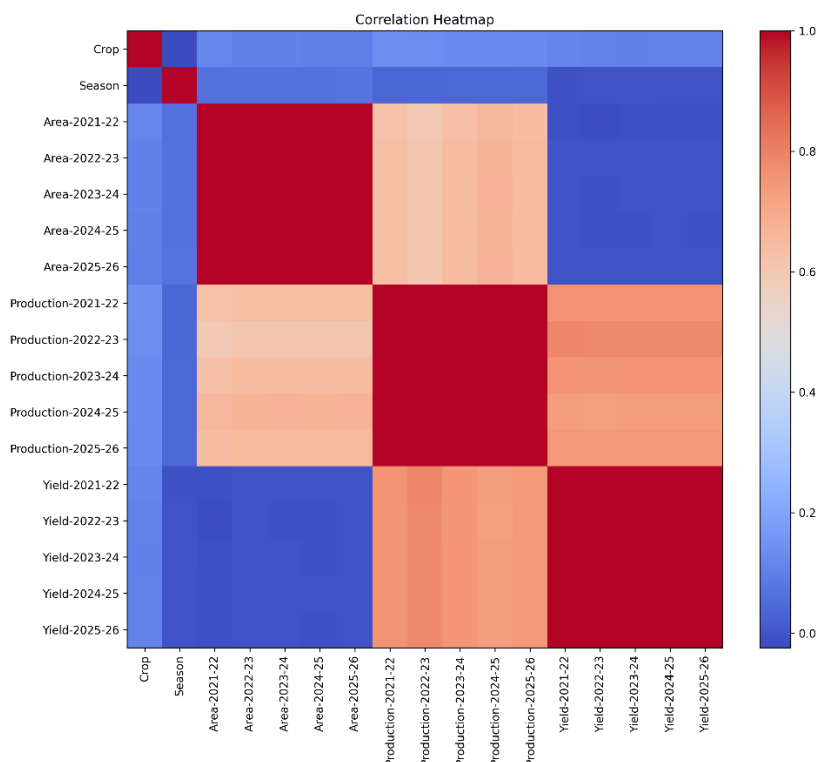


Figure-8 Correlation Heatmap

In the Figure-8, correlation heatmap illustrates the relationships among numerical features in the dataset. Strong positive correlations are observed between area, production, and yield values recorded across different years.

6.2 Model Performance

Model	R-Square Score	MAE	MSE	RMSE
Linear Regression	-2.4747	1327.05	2,633,552.90	1622.82
Decision Tree	0.6842	311.20	239,383.60	489.27
Random Forest Regressor	0.8465	263.34	116,306.90	341.04
Gradient Boosting Regressor	0.9175	206.36	62,539.85	250.08

Table-1 Model Comparison table.

Table 1 compares the performance of the four regression models using standard evaluation metrics. Among all the models, Gradient Boosting Regressor achieved the highest prediction accuracy with the best R^2 Score and the lowest prediction errors.

6.3 Prediction Results

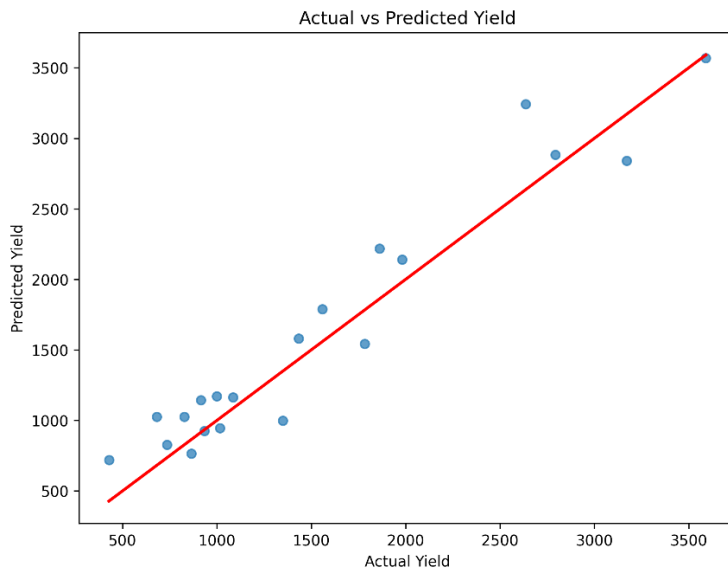


Figure-9 Actual vs Predicted Yield

In the Figure 9, it compares the actual crop yield values with the predicted values generated by the selected Machine Learning model. The close alignment of most points with the reference line indicates that the model provides accurate crop yield predictions.

6.4 Error Analysis

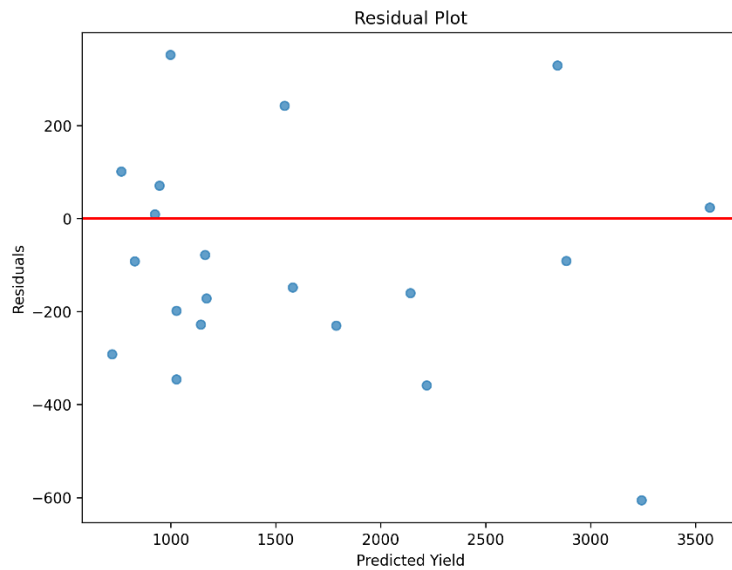


Figure-10 Residual Plot

In the Figure 10, it shows the residuals (prediction errors) against the predicted crop yield values. Most residuals are distributed around the zero line without a clear pattern, indicating that the model has good predictive performance with relatively small errors.

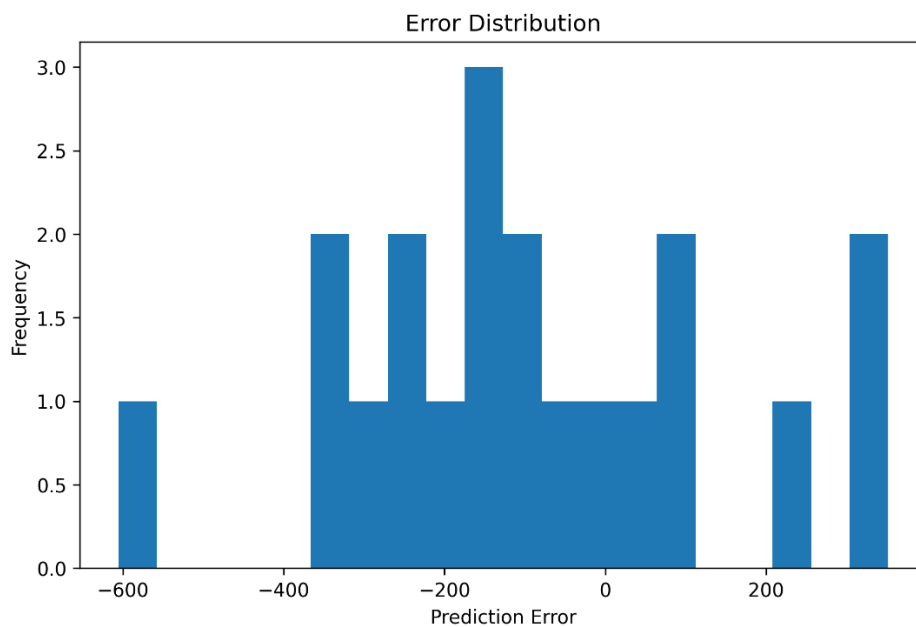


Figure-11 Error Distribution

In the Figure 11, it presents the distribution of prediction errors made by the model. Most errors are concentrated near zero, suggesting that the selected model produces reliable and consistent predictions for crop yield.

7. Conclusion

This project successfully analysed the All-India Crop-wise Area, Production & Yield dataset using Machine Learning techniques to predict crop yield. The dataset was pre-processed, explored through visualizations, and used to train and evaluate four regression models. Among them, the Gradient Boosting Regressor achieved the best performance with an R^2 Score of 0.9175, demonstrating its effectiveness for crop yield prediction on the given dataset.

The project highlights how Artificial Intelligence and Machine Learning can be applied to historical agricultural data to support data-driven decision-making. Although the dataset has certain limitations, such as a small number of records and the absence of weather and soil-related information, the results demonstrate the potential of AI-based approaches for agricultural analysis and crop yield prediction.

8. References

- [1] Department of Agriculture & Farmers Welfare, Government of India. *All India Crop-wise Area, Production & Yield Dataset*. Government of India.
- [2] Government of India. *Open Government Data (OGD) Platform India*. <https://www.data.gov.in/>
- [3] Jeong, J. H., Resop, J. P., Mueller, N. D., Fleisher, D. H., Yun, K., Butler, E. E., Timlin, D. J., Shim, K. M., Gerber, J. S., Reddy, V. R., & Kim, S. H. (2016). *Random Forests for Global and Regional Crop Yield Predictions*. PLoS ONE, 11(6).
- [4] Chlingaryan, A., Sukkarieh, S., & Whelan, B. (2018). *Machine Learning Approaches for Crop Yield Prediction and Nitrogen Status Estimation in Precision Agriculture: A Review*. Computers and Electronics in Agriculture, 151, 61–69.
- [5] Elavarasan, D., & Vincent, D. R. (2020). *Crop Yield Prediction Using Deep Reinforcement Learning Model for Sustainable Agrarian Applications*. IEEE Access, 8, 86886–86901.
- [6] Khaki, S., Wang, L., & Archontoulis, S. V. (2020). *A CNN-RNN Framework for Crop Yield Prediction*. Frontiers in Plant Science, 10.
- [7] Shahhosseini, M., Hu, G., & Archontoulis, S. V. (2021). *Forecasting Corn Yield With Machine Learning Ensembles*. Scientific Reports, 11.
- [8] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.

- [9] Friedman, J. H. (2001). *Greedy Function Approximation: A Gradient Boosting Machine*. The Annals of Statistics, 29(5), 1189–1232.
- [10] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

Appendix

Additional material such as sample outputs, screenshots, or code snippets.

```
Step 1. Data Understanding

# Import libraries
import pandas as pd
import numpy as np

# Load dataset

file_path = "/content/drive/MyDrive/ITI_Ropar_2026/All-India_-Crop-wise-Area,-Production-&-Yield.csv"

df = pd.read_csv(file_path)

# Display first 5 rows

print("First 5 Rows:")
display(df.head())

# Check dataset shape
|
print("\nDataset Shape:")
print(df.shape)

# Check column names

print("\nColumn Names:")
print(df.columns.tolist())

# Check dataset information

print("\nDataset Information:")
df.info()

# Check data types

print("\nData Types:")
print(df.dtypes)

# Check missing values

print("\nMissing Values:")
print(df.isnull().sum())

# Check duplicate rows

print("\nDuplicate Rows:")
print(df.duplicated().sum())

# Display summary statistics for numerical columns

print("\nSummary Statistics:")
display(df.describe())

# Display summary statistics for all columns

print("\nComplete Summary:")
display(df.describe(include="all"))

First 5 Rows:
```

	Crop	Season	Area-2021-22	Area-2022-23	Area-2023-24	Area-2024-25	Area-2025-26	Production-2021-22	Production-2022-23	Production-2023-24	Production-2024-25	Production-2025-26	Yield-2021-22	Yield-2022-23	Yield-2023-24	Yield-2024-25	Yield-2025-26
0	Rice	Kharif	410.38	404.03	407.34	434.66	446.70	1110.01	1105.12	1132.59	1227.72	1247.58	2705.0	2735.0	2780.0	2825.0	2793.0
1	Rice	Rabi	52.40	43.28	40.76	44.73	48.99	184.71	150.04	146.01	161.29	181.07	3525.0	3467.0	3582.0	3606.0	3696.0
2	Rice	Summer	NaN	31.01	30.18	33.28	32.63	NaN	102.40	99.65	112.84	111.59	NaN	3302.0	3302.0	3391.0	3419.0
3	Rice	Total	462.79	478.32	478.28	512.67	528.32	1294.71	1357.55	1378.25	1501.84	1540.24	2798.0	2838.0	2882.0	2929.0	2915.0
4	Wheat	Rabi	304.59	314.01	318.33	328.04	336.00	1077.42	1105.54	1132.92	1179.45	1206.57	3537.0	3521.0	3559.0	3595.0	3591.0

```
Dataset Shape:
(98, 17)

Column Names:
['Crop', 'Season', 'Area-2021-22', 'Area-2022-23', 'Area-2023-24', 'Area-2024-25', 'Area-2025-26', 'Production-2021-22', 'Production-2022-23', 'Production-2023-24', 'Production-2024-25', 'Production-2025-26', 'Yield-2021-22', 'Yield-2022-23', 'Yield-2023-24', 'Yield-2024-25', 'Yield-2025-26']

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 98 entries, 0 to 97
Data columns (total 17 columns):
#   Column              Non-Null Count  Dtype  
```

Duplicate Rows:
0

Summary Statistics:

	Area-2021-22	Area-2022-23	Area-2023-24	Area-2024-25	Area-2025-26	Production-2021-22	Production-2022-23	Production-2023-24	Production-2024-25	Production-2025-26	Yield-2021-22	Yield-2022-23	Yield-2023-24	Yield-2024-25	Yield-2025-26
count	82.000000	98.000000	98.000000	98.000000	98.000000	82.000000	98.000000	98.000000	98.000000	98.000000	82.000000	98.000000	98.000000	98.000000	98.000000
mean	122.503659	104.8950	104.996122	108.247347	111.609388	385.540366	344.773265	337.919898	355.841531	378.725714	3627.329268	3380.591837	3277.112245	3399.683673	3462.561224
std	220.258767	207.8563	208.573919	217.612252	224.955373	863.982860	866.353340	831.040488	860.321993	926.191402	12968.894545	11653.004682	11030.949135	11656.926455	11877.772415
min	0.380000	0.1600	0.320000	0.300000	0.230000	0.330000	0.130000	0.270000	0.190000	0.310000	337.000000	325.000000	323.000000	394.000000	382.000000
25%	8.310000	6.1900	5.605000	5.605000	5.830000	13.982500	10.610000	8.985000	10.515000	10.127500	861.000000	898.000000	884.250000	925.250000	919.750000
50%	47.665000	26.4050	27.360000	27.435000	29.430000	90.980000	34.940000	34.170000	40.455000	44.240000	1307.000000	1369.000000	1356.000000	1420.500000	1401.000000
75%	121.470000	103.8975	97.977500	92.922500	98.107500	217.775000	156.947500	150.480000	163.232500	179.762500	2483.500000	2611.250000	2595.500000	2609.250000	2711.250000
max	1301.690000	1322.0400	1321.040000	1375.460000	1423.250000	4394.250000	4905.330000	4531.580000	4546.110000	5000.630000	84906.000000	83349.000000	78953.000000	83416.000000	84987.000000

Complete Summary:

	Crop	Season	Area-2021-22	Area-2022-23	Area-2023-24	Area-2024-25	Area-2025-26	Production-2021-22	Production-2022-23	Production-2023-24	Production-2024-25	Production-2025-26	Yield-2021-22	Yield-2022-23	Yield-2023-24	Yield-2024-25	Yield-2025-26
count	98	98	82.000000	98.000000	98.000000	98.000000	98.000000	82.000000	98.000000	98.000000	98.000000	98.000000	82.000000	98.000000	98.000000	98.000000	98.000000
unique	34	4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
top	Rice	Total	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
freq	4	34	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
mean	NaN	NaN	122.503659	104.8950	104.996122	108.247347	111.609388	385.540366	344.773265	337.919898	355.841531	378.725714	3627.329268	3380.591837	3277.112245	3399.683673	3462.561224
std	NaN	NaN	220.258767	207.8563	208.573919	217.612252	224.955373	863.982860	866.353340	831.040488	860.321993	926.191402	12968.894545	11653.004682	11030.949135	11656.926455	11877.772415
min	NaN	NaN	0.380000	0.1600	0.320000	0.300000	0.230000	0.330000	0.130000	0.270000	0.190000	0.310000	337.000000	325.000000	323.000000	394.000000	382.000000
25%	NaN	NaN	8.310000	6.1900	5.605000	5.605000	5.830000	13.982500	10.610000	8.985000	10.515000	10.127500	861.000000	898.000000	884.250000	925.250000	919.750000
50%	NaN	NaN	47.665000	26.4050	27.360000	27.435000	29.430000	90.980000	34.940000	34.170000	40.455000	44.240000	1307.000000	1369.000000	1356.000000	1420.500000	1401.000000
75%	NaN	NaN	121.470000	103.8975	97.977500	92.922500	98.107500	217.775000	156.947500	150.480000	163.232500	179.762500	2483.500000	2611.250000	2595.500000	2609.250000	2711.250000
max	NaN	NaN	1301.690000	1322.0400	1321.040000	1375.460000	1423.250000	4394.250000	4905.330000	4531.580000	4546.110000	5000.630000	84906.000000	83349.000000	78953.000000	83416.000000	84987.000000

Step 2 Data cleaning

```
# Create a copy of the dataset
df_clean = df.copy()

# Check missing values before cleaning
print("Missing Values Before Cleaning:")
print(df_clean.isnull().sum())

# Check duplicate rows before cleaning
print("\nDuplicate Rows Before Cleaning:")
print(df_clean.duplicated().sum())

# Remove duplicate rows
df_clean = df_clean.drop_duplicates()

# Convert numeric columns
numeric_columns = [
    "Area-2021-22", "Production-2021-22", "Yield-2021-22",
    "Area-2022-23", "Production-2022-23", "Yield-2022-23",
    "Area-2023-24", "Production-2023-24", "Yield-2023-24",
    "Area-2024-25", "Production-2024-25", "Yield-2024-25",
    "Area-2025-26", "Production-2025-26", "Yield-2025-26"
]

for col in numeric_columns:
    df_clean[col] = pd.to_numeric(df_clean[col], errors="coerce")

# Fill missing numeric values with the column median
df_clean[numeric_columns] = df_clean[numeric_columns].fillna(
    df_clean[numeric_columns].median()
)
```

```
[ ] 1 print("\nFirst 5 Rows of Cleaned Dataset:")
    display(df_clean.head())
```

```
...
Missing Values After Cleaning:
Crop          0
Season        0
Area-2021-22  0
Area-2022-23  0
Area-2023-24  0
Area-2024-25  0
Area-2025-26  0
Production-2021-22  0
Production-2022-23  0
Production-2023-24  0
Production-2024-25  0
Production-2025-26  0
Yield-2021-22  0
Yield-2022-23  0
Yield-2023-24  0
Yield-2024-25  0
Yield-2025-26  0
dtype: int64

Cleaned Dataset Shape:
(98, 17)

Data Types After Cleaning:
Crop          int64
Season        int64
Area-2021-22  float64
Area-2022-23  float64
Area-2023-24  float64
Area-2024-25  float64
Area-2025-26  float64
Production-2021-22  float64
Production-2022-23  float64
Production-2023-24  float64
Production-2024-25  float64
Production-2025-26  float64
Yield-2021-22  float64
Yield-2022-23  float64
Yield-2023-24  float64
Yield-2024-25  float64
Yield-2025-26  float64
```

Step 3. Exploratory Data Analysis (EDA)

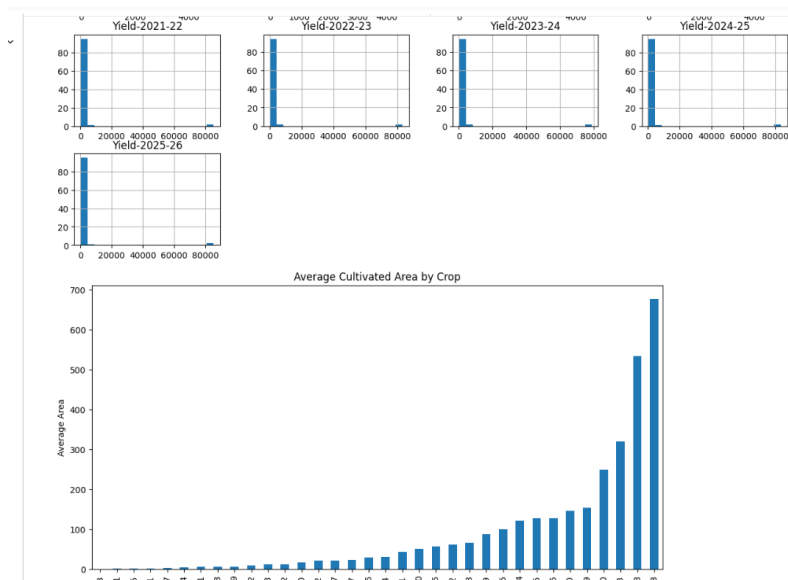
```
[ ] 1 # Import libraries
    import os
    import pandas as pd
    import matplotlib.pyplot as plt
    from google.colab import drive

    # Mount Google Drive
    drive.mount("/content/drive")

    # Create results folder
    results_path = "/content/drive/MyDrive/IIIT_Ropar_2026/IIIT_Ropar_2026/results"
    os.makedirs(results_path, exist_ok=True)

    # Crop distribution
    plt.figure(figsize=(10,5))
    df_clean["Crop"].value_counts().plot(kind="bar")
    plt.title("Crop Distribution")
    plt.xlabel("Crop")
    plt.ylabel("Count")
    plt.xticks(rotation=45)
    plt.savefig(f"{results_path}/crop_distribution.png", dpi=300, bbox_inches="tight")
    plt.show()

    # Season distribution
    plt.figure(figsize=(6,5))
    df_clean["Season"].value_counts().plot(kind="bar")
    plt.title("Season Distribution")
    plt.xlabel("Season")
    plt.ylabel("Count")
    plt.savefig(f"{results_path}/season_distribution.png", dpi=300, bbox_inches="tight")
    plt.show()
```



Step 4. Feature Engineering

```
# Import libraries
from sklearn.preprocessing import LabelEncoder

# Create a copy for feature engineering
feature_df = df_clean.copy()

# Encode crop names
crop_encoder = LabelEncoder()
feature_df["Crop"] = crop_encoder.fit_transform(feature_df["Crop"])

# Encode season names
season_encoder = LabelEncoder()
feature_df["Season"] = season_encoder.fit_transform(feature_df["Season"])

# Create a year feature
feature_df["Year"] = 2025

# Select input features
X = feature_df[
    [
        "Crop",
        "Season",
        "Year",
        "Area-2025-26",
        "Production-2025-26"
    ]
]

# Select target variable
y = feature_df["Yield-2025-26"]

# Display feature matrix
print("Feature Matrix")
display(X.head())

# Display target values
```

```
print("Target Shape:", y.shape)
```

Feature Matrix

	Crop	Season	Year	Area-2025-26	Production-2025-26
0	20	0	2025	446.70	1247.58
1	20	1	2025	48.99	181.07
2	20	2	2025	32.63	111.59
3	20	3	2025	528.32	1540.24
4	33	1	2025	336.00	1206.57

Target Variable

	Yield-2025-26
0	2793.0
1	3696.0
2	3419.0
3	2915.0
4	3591.0

dtype: float64
 Feature Shape: (98, 5)
 Target Shape: (98,)

Step 5. Train-Test Split

```
# Import library
from sklearn.model_selection import train_test_split

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# Display dataset shapes
print("Training Features Shape:", X_train.shape)
print("Testing Features Shape:", X_test.shape)

print("Training Target Shape:", y_train.shape)
print("Testing Target Shape:", y_test.shape)

# Display first few training samples
print("\nTraining Features")
display(X_train.head())

print("\nTraining Target")
display(y_train.head())
```

```
... Training Features Shape: (78, 5)
Testing Features Shape: (20, 5)
Training Target Shape: (78,)
Testing Target Shape: (20,)
```

Training Features

	Crop	Season	Year	Area-2025-26	Production-2025-26
49	17	0	2025	15.66	9.46
70	15	0	2025	0.83	0.32
68	22	2	2025	5.04	4.51
15	7	3	2025	35.48	43.56
39	32	0	2025	21.26	11.32

Step 6. Machine Learning Models

```
# Import libraries
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor

from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error

import numpy as np
import pandas as pd

# Create models
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),
    "Gradient Boosting": GradientBoostingRegressor(
        random_state=42
    )
}

# Train and evaluate models
results = []
predictions = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    predictions[name] = y_pred
    r2 = r2_score(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
```

Model Comparison

	Model	R2 Score	MAE	MSE	RMSE
3	Gradient Boosting	0.917485	206.359086	6.253985e+04	250.079690
2	Random Forest	0.846545	263.337500	1.163069e+05	341.038021
1	Decision Tree	0.684158	311.200000	2.393836e+05	489.268433
0	Linear Regression	-2.474701	1327.046259	2.633553e+06	1622.822591

Best Model: Gradient Boosting
Best R2 Score: 0.9175

Next steps: [Generate code with results_df](#) [New interactive sheet](#)

Step 7. Model Evaluation

```
# Display model evaluation results
```

```
print("Model Evaluation Results")
display(results_df)

# Select the best model
best_model_index = results_df["R2 Score"].idxmax()
best_model_name = results_df.loc[best_model_index, "Model"]
best_r2 = results_df.loc[best_model_index, "R2 Score"]
best_mae = results_df.loc[best_model_index, "MAE"]
best_mse = results_df.loc[best_model_index, "MSE"]
best_rmse = results_df.loc[best_model_index, "RMSE"]

# Print best model performance
print("\nBest Performing Model")
print(f"Model : {best_model_name}")
print(f"R² Score : {best_r2:.4f}")
print(f"MAE : {best_mae:.4f}")
print(f"MSE : {best_mse:.4f}")
print(f"RMSE : {best_rmse:.4f}")
```

Model Evaluation Results

	Model	R2 Score	MAE	MSE	RMSE
3	Gradient Boosting	0.917485	206.359086	6.253985e+04	250.079690
2	Random Forest	0.846545	263.337500	1.163069e+05	341.038021
1	Decision Tree	0.684158	311.200000	2.393836e+05	489.268433
0	Linear Regression	-2.474701	1327.046259	2.633553e+06	1622.822591

Best Performing Model
Model : Gradient Boosting
R² Score : 0.9175
MAE : 206.3591
MSE : 62539.85
RMSE : 250.0797

Step 8. Visualizations

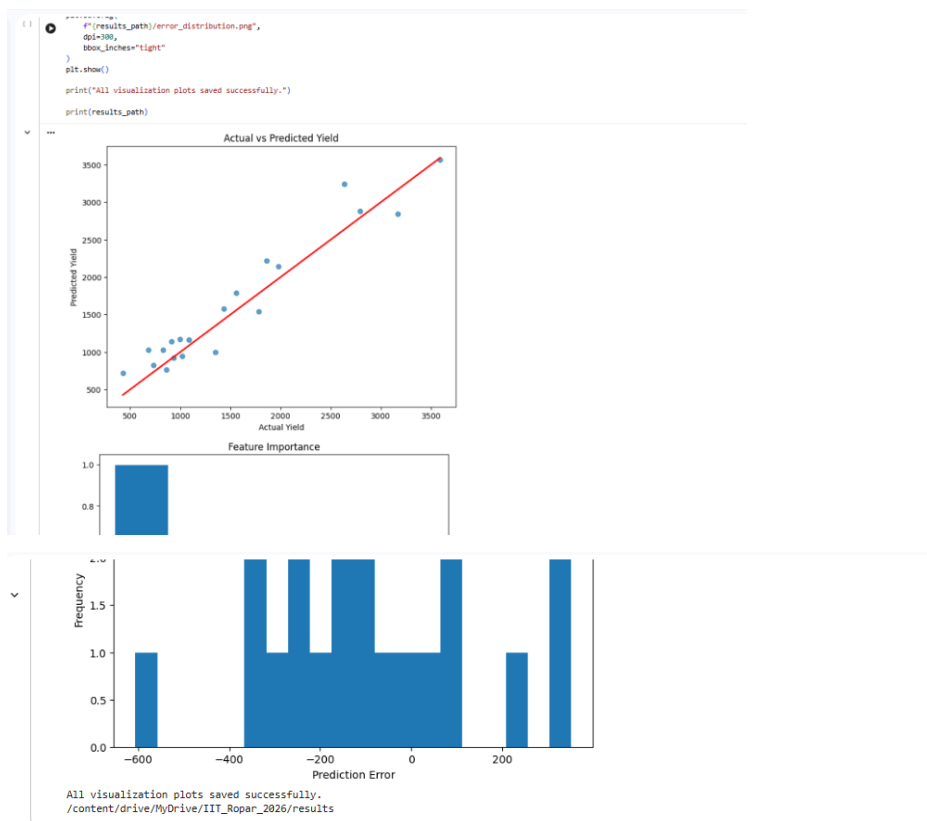
```
# Import libraries
import os
import pandas as pd
import matplotlib.pyplot as plt

# Create results folder
results_path = "/content/drive/MyDrive/IIT_Ropar_2026/results"
os.makedirs(results_path, exist_ok=True)

# Predict using the best model
y_pred = best_model.predict(X_test)

# Actual vs Predicted Yield
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red",
    linewidth=2
)
plt.title("Actual vs Predicted Yield")
plt.xlabel("Actual Yield")
plt.ylabel("Predicted Yield")
plt.savefig(
    f"{results_path}/actual_vs_predicted_yield.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()

# Feature importance
if hasattr(best_model, "feature_importances_"):
    feature_importance = pd.DataFrame({
        "Feature": X.columns,
        "Importance": best_model.feature_importances_
    })
```



Step 9- Model Saving

```

1 # Import libraries
2 import os
3 import joblib
4
5 # Create models folder
6 model_path = "/content/drive/MyDrive/IIT_Ropar_2026/models"
7 os.makedirs(model_path, exist_ok=True)
8
9 # Save the best model
10 joblib.dump(
11     best_model,
12     f"{model_path}/best_crop_yield_model.pkl"
13 )
14 print("Best model saved successfully.")
15 print(model_path)

```

Best model saved successfully.
/content/drive/MyDrive/IIT_Ropar_2026/models

Google Colab Link:

<https://colab.research.google.com/drive/1lJsWDK4Zial4Zf5eyzKTABF7TesPyQYG?usp=sharing>

Git-Hub Repository Link:

https://github.com/Rushikannan2/AI_Agriculture_Project